Nightfall Labs: Team Deliverable 3

A. Requirements

The scope for Phase 1 remains mostly the same as described in Deliverable 2, but the requirements in this section are written in more detail to reflect the current implementation focus. Phase 1 is centered on the core ordering flow of the system. The goal of this phase is to allow a user to browse menu content, manage a cart, enter checkout information, submit an order, store the order, and view confirmation and order history information. Advanced features planned for later phases, such as ranked search, allergen warning logic, delivery radius enforcement, and email notifications, are not the main focus of this implementation stage.

<u>FR1: Menu Loading</u>
The system loads and displays menu items to the user when the menu page is opened so the user can begin browsing available food options.

<u>FR2: Category Grouping</u>
The system organizes menu items by category so users can more easily browse types of items such as burgers, wraps, or salads.

<u>FR3: Item Details</u>
The system allows the user to view item information, including the item name, description, price, ingredients, allergens, and availability information when that item is selected.

<u>FR4: Add to Cart</u>
The system allows the user to add a selected menu item to the shopping cart for later checkout.

FR5: Modify Cart
The system allows the user to update cart contents by changing quantities or removing items before submitting an order.

FR6: Order Calculations
The system calculates and displays the subtotal, tax, delivery fee, service fee, optional tip, and final total for the order.

FR7: Cart Integrity
The system prevents the user from checking out if no items are currently in the cart.

FR8: Information Collection
The system collects checkout information needed to place an order, including pickup or delivery selection, contact email, payment name, card number, and delivery address when applicable.

FR9: Field Validation
The system validates required checkout fields before an order is submitted. This includes checking for missing values and validating that the credit card number is in the correct format.

FR10: Payment Transmission
The system submits order and payment related information through the application workflow so the order can be processed.

FR11: Order ID Generation
The system generates a unique order identifier after a successful order submission.

FR12: Order Storage
The system stores completed order information in the backend database so that it can be retrieved later.
FR13: Confirmation Display
The system displays a confirmation page after a successful order is placed and shows the generated order ID to the user.

B. All non-functional requirements (NFRs)

Product Requirements

Performance
NFR : Search Response Time
Search results shall be returned within 1 second for a menu containing up to 500 items.
NFR : Checkout Processing Time
Checkout confirmation shall be displayed within 3 seconds of receiving a successful payment response.
Reliability
NFR : Order Integrity
The system shall prevent duplicate order creation during network interruptions.
NFR : Availability
The system shall maintain 99% availability during operating hours.
Security
NFR : Encryption
All client-server communication shall use HTTPS encryption.
NFR : Payment Data Protection
The system shall not store raw credit card information.
NFR : Input Validation
The system shall validate and sanitize all user inputs.
Usability
NFR : Responsive Design
The system shall support mobile, tablet, and desktop screen sizes.
NFR : Checkout Simplicity
Users shall be able to complete checkout in no more than five steps.
NFR : Accessibility Compliance
The system shall comply with WCAG 2.1 Level AA accessibility standards.

Organizational Requirements
NFR : Development Standards
The system shall be developed using RESTful API architecture.
NFR : Documentation
All deliverables shall conform to ITIS 3300 documentation standards.
NFR : Maintainability
The system architecture shall be modular to allow future feature enhancements.

External Requirements

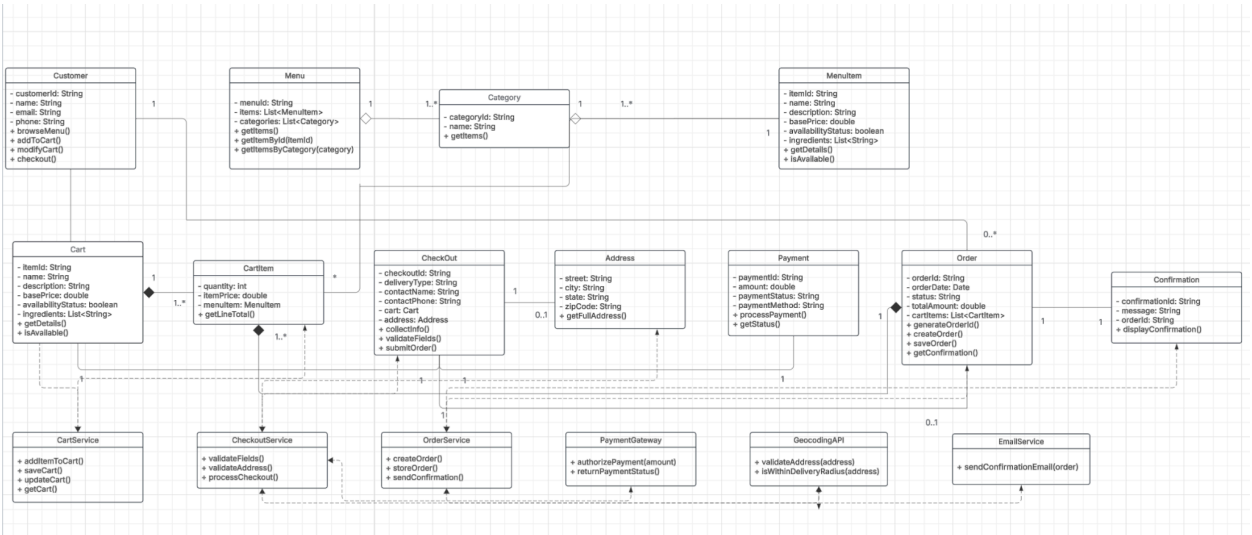
Interoperability
NFR : Payment Integration
The system shall integrate with an external payment processing API.
NFR : Geocoding Integration
The system shall integrate with a geocoding API for address validation.

Privacy & Legislative
NFR : Data Privacy
The system shall allow users to review and correct their stored contact information.
NFR : Data Protection Compliance
The system shall comply with applicable data protection regulations.

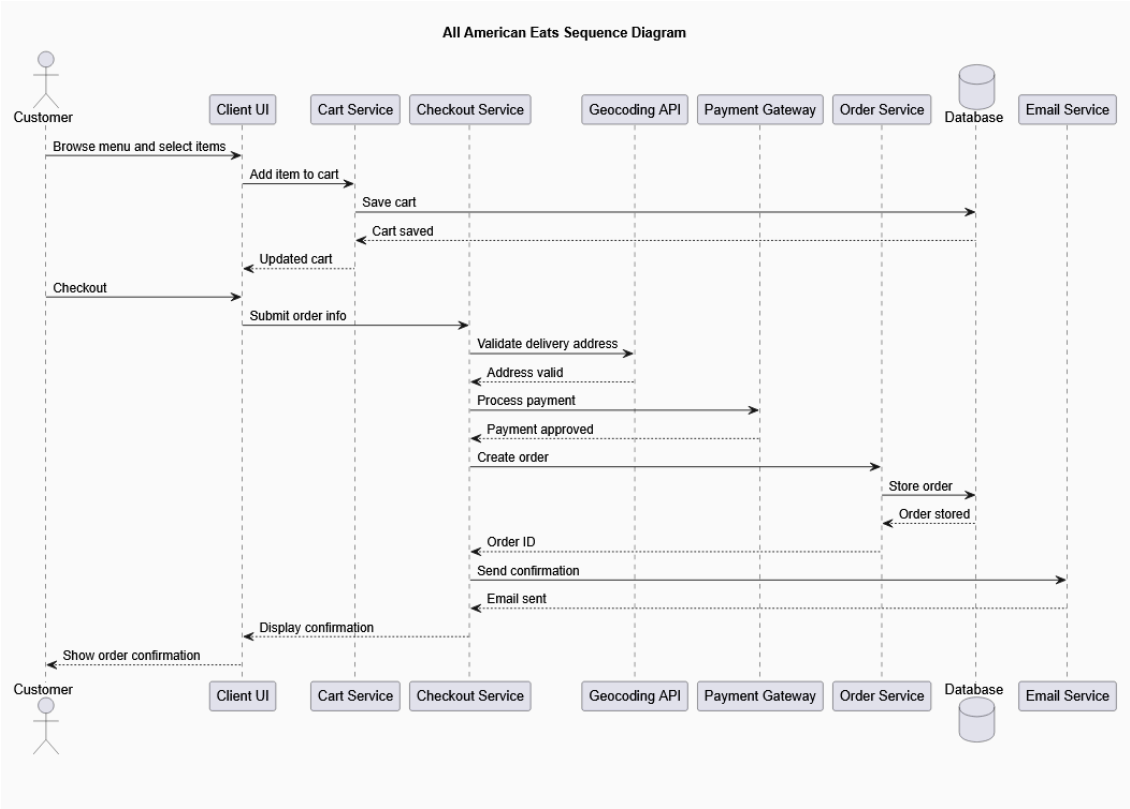
b. UML design for phase 1

You must include the following diagrams:

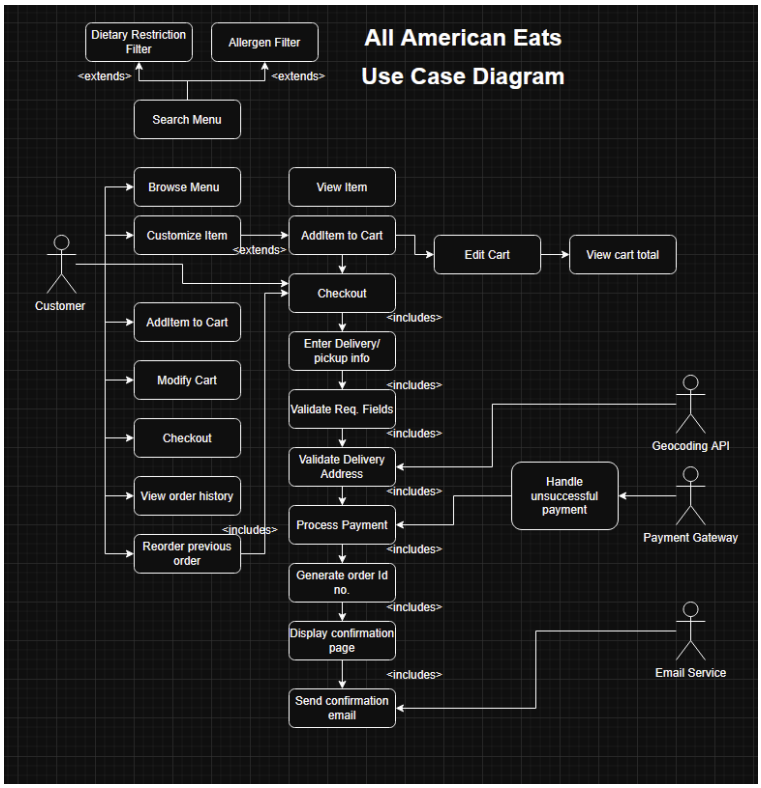
- **Class diagram**



• Sequence diagram



• Use case diagram



C. User Manual (How to install/use the program)

OVERVIEW:
Welcome to the All American Eats ordering application! This system allows users to browse our delicious menu, filter items based on their preferences, add selections to a cart, and place an order for delivery or pickup.

INSTALLATION (End User instructions):
For end-users, there is no installation required. Simply open your modern web browser (Google Chrome, Firefox, Edge) and navigate to the application URL provided by the restaurant (locally, this will be your `index.html` file).

HOW TO USE:
1. Browsing the Menu:
   - Navigate to the Home page ('Menu' tab).
   - Use the category filters on the left sidebar to show only specific types of items (such as Burgers, Wraps, or Salads).
   - Use the dietary restrictions checkboxes or allergen exclusions to filter out unwanted items.
   - Use the top search bar to look for a specific craving you have.
2. Adding to Cart:
   - Click on an item to view its details.
   - Add it to your cart (and select any customizations if applicable).
3. Viewing your Cart:
   - Navigate to the 'Cart' page using the top navigation bar.
   - Here you can review your items, subtotal, tax, fees, and adjust your tip.
4. Checkout:
   - Proceed to checkout from the cart page.
   - Select whether you want "Delivery" or "Pickup".
   - Enter your email, cardholder name, and 16-digit card number.
   - If "Delivery" is selected, enter your delivery address.
   - Click "Place Order" to finalize your purchase!
5. Viewing Past Orders:
   - Navigate to the 'Orders' page in the top menu to view your localized order history and totals.

D. Instructions on How to Compile/Run the Program and Test Cases

SYSTEM PREREQUISITES:
- Node.js (version 14.x or higher) installed. This allows the backend server to run.
- NPM (Node Package Manager).

COMPILING & RUNNING THE APPLICATION:
1. Setting up the Backend:
   - Open a command line interface (Terminal/Command Prompt) in the root directory of the project.
   - Run the following command to download the server dependencies (reads from package.json):
     `npm install`
   - Change directories to the backend folder and start the application server:
     `cd src/backend`
     `node server.js`
   - You should see the message: "Connected to database" and "Server running on http://localhost:3000". Leave this terminal window open.

2. Running the Frontend:
   - Open the directory `src/frontend` in your file explorer.
   - Double-click on `index.html` to open the application in your default web browser.
   - Ensure the server is actively running in the background for database interactions (like adding cart items or retrieving orders) to succeed.

RUNNING TEST CASES:
Currently, the application relies on manual acceptance testing to verify functionality.
- Open the application in your browser (`index.html`).
- Perform the following manual test scenarios to verify functionality:
  - Test 1 (Menu filters): Check all dietary checkboxes and confirm only compliant menu items appear.
  - Test 2 (Cart Validation): Attempt to proceed to checkout with an empty cart (the application should prevent this).

- Test 3 (Checkout form): Enter an invalid credit card number during checkout and verify that it requires exactly 16 digits.
- Test 4 (Order lifecycle): Complete a successful order, go to the "Orders" page, and ensure the new order ID correctly displays alongside the correct total.

E. Reflection on Accomplishments, Successes, and Areas for Improvement

WHAT HAS BEEN ACCOMPLISHED:

During this phase, we successfully created the foundation of a complete restaurant ordering architecture. We built a fully functional client-server relationship with a frontend written in HTML/JS/CSS and a Node.js Express backend communicating with a local SQLite database. We achieved our objectives of creating a structured menu view, a dynamic cart calculation system, and a functional checkout process that logs orders into a database table.

WHAT WENT WELL:

- Backend Architecture: Setting up Express and SQLite 3 was extremely efficient, and creating the API endpoints (`GET` and `POST` for orders) went smoothly, providing immediate connectivity for the frontend forms.
- Dynamic Form Handling: The frontend UI gracefully controls form requirements (such as optionally asking for a delivery address based on a dropdown selection) making for a seamless user experience.
- File modularity: Breaking down the frontend pages into specific chunks (`index`, `cart`, `checkout`, `orders`, `confirmation`) and pairing them with specific logic files (`Checkout.js`, `Orders.js`) kept the codebase organized.

WHAT COULD BE IMPROVED:

- Visual Polish and Modals: While our HTML structure and Javascript hookups exist for the customization modals, more time is needed to polish the CSS and visual presentation of the front end.

- Automated Testing: Our current testing strategy relies entirely on manual verification. In future iterations, we want to integrate an automated testing framework like Jest for the backend and Selenium/Cypress for the frontend.
- Error Handling: Although the application returns 500/400 errors from the server if database calls fail, we could do more to display user-friendly error banners directly in the client UI instead of having them die silently or in the browser-console dependent.

f. Member contribution table

Member name	Contribution description	Overall Contribution (%)	Note (if applicable)
Thomas Clayton	-Created the use case diagram. -Created the sequence diagram. -(Front end) index.html -(Front end) cart.html -(Front end) checkout.html -(Front end) orders.html -(Front end) confirmation.html	40%	
Isaac Baraki	-Created class UML diagram -Worked on use case diagram -Worked on sequence diagram	15%	
Alex Lopez Robles	(Backend) Order.js (Backend) Checkout.js (Backend) Confirmation.js - Updated Core	15%	

	Functionalities based on backend work.		
Danny Moncada	- Revised and expanded Part A of the report to better reflect Phase 1 requirements for Deliverable 3, helped review the report for consistency with the current implementation, and assisted with final deliverable cleanup and organization.	15%	
Justus Purnell			
Jayant Thathireddy	- Built menu, filtering logic, cart state management, database seeding, and form validation - Authored the user manual, compilation /execution instructions, and project reflection sections for Team Deliverable 3 - Standardized the README, gitignore, and dependency control in the repo	15%	